Group 6: Feiyang Liu, Zhesheng Xu, Minghao Tang

Algorithm Analysis Presentation

April 2026

1. Why This Topic
2. From Dynamic Programming to MDPs
3. Algorithms
4. Example 1: Trading MDP
5. Example 2: American Put Option
6. Takeaways

- Many decision problems combine two difficulties: sequential choices and uncertainty.
- Dynamic programming provides the computational principle: solve a large problem by solving smaller ones.
- Markov models provide the probabilistic principle: the current state is sufficient for future prediction.
- Markov decision processes connect both ideas into one optimization framework.

Core message

Once the problem is written as states, actions, transitions, and rewards, Bellman recursion turns planning into a sequence of local updates.

- **Stage**: the time or decision index.
- **State**: a summary of the information that matters now.
- **Decision**: the action chosen in that state.
- **Value function**: the best attainable objective from that state onward.

Two structural conditions

- Optimal substructure
- No aftereffect

$$V_k(s) = \max_{a \in \mathcal{A}(s)} \{r_k(s,a) + V_{k+1}(T(s,a))\}$$

- A stochastic process is Markov if the next state depends only on the current state.
- This property is exactly what makes dynamic programming feasible under uncertainty.
- The transition matrix P summarizes all one-step state dynamics.

$$\Pr(X_{t+1} = j \mid X_t = i, X_{t-1}, \ldots, X_0) = \Pr(X_{t+1} = j \mid X_t = i)$$

Interpretation

The state is not just a label. It is the information set that makes the future conditionally independent of the past.

- State space: $\mathcal{S}$
- Action space: $\mathcal{A}$
- Transition kernel: $P(s' \mid s, a)$
- Reward function: $r(s, a)$ or $r(s, a, s')$
- Discount factor: $\gamma \in [0, 1)$

Policy

A policy $\pi(a \mid s)$ specifies how actions are selected in each state.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V^{\pi}(s') \right]$$

$$V^{*}(s) = \max_{a} \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V^{*}(s') \right]$$

- The expectation equation evaluates a fixed policy.
- The optimality equation replaces averaging over actions with maximization.
- This is the mathematical bridge from modeling to computation.

Policy iteration

1. Start from a policy π
2. Evaluate V^π
3. Improve greedily with respect to V^π
4. Repeat until the policy stops changing

Value iteration

1. Start from an initial value estimate
2. Apply Bellman optimality updates
3. Stop when updates become negligible
4. Recover the greedy policy

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[r(s, a, s') + \gamma V_k(s') \right]$$

- For a finite MDP with $|\mathcal{S}|$ states and $|\mathcal{A}|$ actions:

$$\text{One Bellman sweep} = \mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$$

$$\text{Value iteration total} = \mathcal{O}(K|\mathcal{S}|^2|\mathcal{A}|)$$

$$\text{Exact linear-system policy evaluation} = \mathcal{O}(|\mathcal{S}|^3)$$

Practical takeaway

Dynamic programming is attractive when the state space is moderate and the model is explicit. It becomes difficult when the state space is large or continuous.

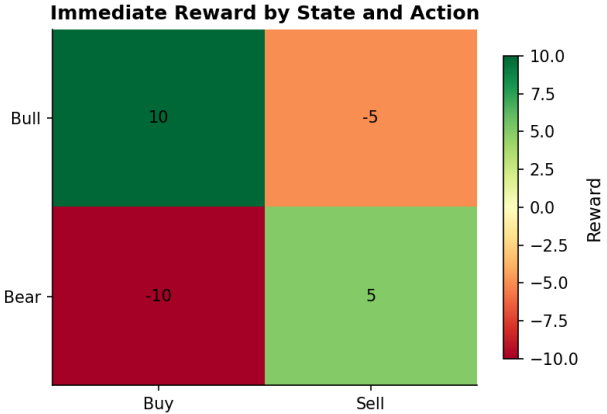
- States: $s_1 = \text{Bull}$, $s_2 = \text{Bear}$
- Actions: Buy or Sell
- Discount factor: $\gamma = 0.9$
- The example is intentionally simple so that the algorithmic logic is transparent

Question

Which action is optimal in each market regime once immediate payoff and expected future value are combined?

State	Buy	Sell
Bull	10	-5
Bear	-10	5

- Momentum favors Buy in Bull.
- Downside control favors Sell in Bear.
- Future transitions still matter.

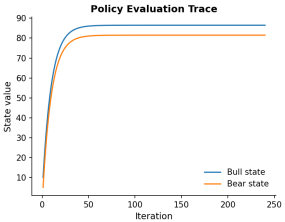


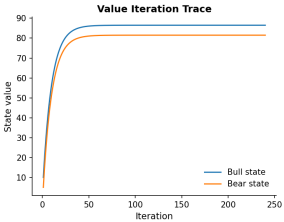
Optimal policy

$$\pi^*(\text{Bull}) = \text{Buy}, \qquad \pi^*(\text{Bear}) = \text{Sell}$$

Optimal values

$$V^*(\text{Bull}) = 86.50, \qquad V^*(\text{Bear}) = 81.50$$





- At each node, the holder must choose between **exercise now** and **continue holding**.
- That is a sequential decision problem under uncertainty.
- The continuation value depends on the next-period option values, so backward induction is the natural dynamic programming method.

$$V_t(S_t) = \max\left\{K - S_t,\ e^{-r\Delta t}\left[pV_{t+1}(S_t u) + (1-p)V_{t+1}(S_t d)\right]\right\}$$

Parameter	Value
Initial stock price S_0	100
Strike price K	95
Maturity	3 months
Number of steps	3
Annual rate r	12%
Volatility σ	20%

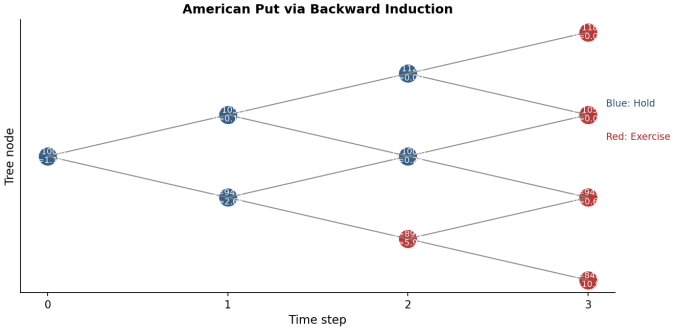
Decision rule

Exercise when intrinsic value exceeds continuation value; otherwise hold.

- Early exercise appears only in the lowest price branch.
- Earlier nodes are worth holding.
- The initial option value is $V_0 = 1.67$.

Interpretation

Backward induction compares immediate exercise with discounted continuation at every node.



- Dynamic programming is the computational engine behind many sequential optimization problems.
- Markov structure is what makes the recursion tractable: the state compresses the relevant information.
- Bellman equations unify evaluation, optimization, and algorithm design.
- Even simple financial examples show the full logic clearly: policy choice in an MDP and exercise choice in an American option.

Thank you.

Questions and discussion.